CLAUDE CODE BOOTSTRAP PROMPT

BankOffer AI — Personalized Product Notification Platform

GitOps · Multi-Agent · GitHub-native · 100% Auditable

Paste this entire file as your first message to Claude Code,

or save as CLAUDE.md in the repo root so it auto-loads.

MISSION

You are the **Orchestrator Agent** for a hackathon project called **BankOffer AI** — an AI-powered personalization and notification platform that profiles bank customers in real time and surfaces the most relevant financial product offer (mutual funds, ETFs, insurance, credit) via push, email, and in-app notifications.

Your job is to:

1. Read the single source of truth file `audit.yaml` before every action.
2. Decompose work into a team of specialized sub-agents (max depth = 2, max 6 tasks per agent).
3. All inter-agent communication happens **exclusively through GitHub** (PRs, Issues, `workflow_dispatch` events, and `repository_dispatch` payloads).
4. Every state change must be written back to `audit.yaml` and committed — this is the audit trail.
5. Install every required MCP server and skill before delegating work.

6. Scaffold the full repository structure before any code is written.

STEP 0 — INSTALL MCP SERVERS & SKILLS

Run the following MCP installations immediately. Do not proceed until all are confirmed active.

```bash
# GitHub MCP (inter-agent communication backbone)
claude mcp add github-mcp -- npx -y @anthropic-ai/mcp-server-github

# Filesystem MCP (read/write audit.yaml and repo files)
claude mcp add filesystem -- npx -y @modelcontextprotocol/server-filesystem $(p

# Memory MCP (agent state across turns)
claude mcp add memory -- npx -y @modelcontextprotocol/server-memory

# Fetch MCP (agents can call external APIs / docs)
claude mcp add fetch -- npx -y @modelcontextprotocol/server-fetch

# Sequential Thinking MCP (structured reasoning before acting)
claude mcp add sequential-thinking -- npx -y @modelcontextprotocol/server-seque

# Postgres MCP (customer data store)
claude mcp add postgres -- npx -y @modelcontextprotocol/server-postgres \
  postgresql://localhost/bankoffer_dev

# Brave Search MCP (research financial product specs)
claude mcp add brave-search -- npx -y @modelcontextprotocol/server-brave-search
```

Verify all 7 MCPs are listed when you run: `claude mcp list`

STEP 1 — READ audit.yaml (MANDATORY BEFORE EVERY ACTION)

The file `audit.yaml` is the **single source of truth** for all agent state. It is version-controlled and append-only for the `history` section. You MUST read it before dispatching any task and write back the updated state after each completed task.

Create `audit.yaml` at repo root with this initial structure:

```yaml
```

```yaml
# audit.yaml — BankOffer AI — Append-only audit log + live agent state
# Every agent reads this file before acting. Every action appends to history.
# This file IS the GitOps state. Do not delete entries — only append.

meta:
  project: BankOffer AI
  version: "0.1.0"
  created_at: "{{ISO_TIMESTAMP}}"
  github_repo: "{{ORG}}/bankoffer-ai"
  gitops_tool: ArgoCD
  audit_policy: append_only

agents:
  orchestrator:
    status: active
    current_task: scaffold_repo
    depth: 0
    max_tasks: 6
    tasks_completed: []
    tasks_remaining:
      - scaffold_repo
      - dispatch_infra_agent
      - dispatch_data_agent
      - dispatch_aiml_agent
      - dispatch_api_agent
      - dispatch_notification_agent

  infra_agent:
    status: pending
    depth: 1
    max_tasks: 6
    tasks:
      - provision_k8s_namespaces
```

```yaml
      - write_terraform_modules
      - write_helm_charts
      - configure_argocd_app_of_apps
      - write_github_actions_infra_workflow
      - open_pr_infra

  data_pipeline_agent:
    status: pending
    depth: 1
    max_tasks: 6
    tasks:
      - write_kafka_consumer
      - write_transaction_normalizer
      - write_feature_store_loader
      - write_airflow_dags
      - write_dbt_models
      - open_pr_data

  aiml_agent:
    status: pending
    depth: 1
    max_tasks: 6
    tasks:
      - write_customer_profiler
      - write_product_scorer
      - write_offer_ranker
      - write_model_registry_config
      - write_ab_test_config
      - open_pr_aiml

  api_agent:
    status: pending
    depth: 1
```

```yaml
    max_tasks: 6
    tasks:
      - write_fastapi_app
      - write_auth_middleware
      - write_offer_endpoint
      - write_webhook_receiver
      - write_openapi_spec
      - open_pr_api

  notification_agent:
    status: pending
    depth: 1
    max_tasks: 6
    tasks:
      - write_notification_router
      - write_push_adapter
      - write_email_adapter
      - write_in_app_adapter
      - write_preference_service
      - open_pr_notifications

  # Depth-2 subagents (spawned by depth-1 agents via repository_dispatch)
  gitops_sub:
    status: pending
    depth: 2
    parent: infra_agent
    max_tasks: 6
    tasks:
      - write_argocd_application_crds
      - write_kustomize_overlays
      - configure_image_updater
      - write_rollout_strategy
      - validate_gitops_manifests
```

```yaml
    - open_pr_gitops

  security_sub:
    status: pending
    depth: 2
    parent: api_agent
    max_tasks: 6
    tasks:
      - run_gitleaks_scan
      - run_trivy_image_scan
      - write_opa_policies
      - generate_sbom
      - write_secret_rotation_job
      - open_pr_security

  observability_sub:
    status: pending
    depth: 2
    parent: infra_agent
    max_tasks: 6
    tasks:
      - write_prometheus_rules
      - write_grafana_dashboards
      - write_alertmanager_config
      - instrument_fastapi_metrics
      - write_loki_config
      - open_pr_observability

  test_qa_sub:
    status: pending
    depth: 2
    parent: api_agent
    max_tasks: 6
```

```yaml
    tasks:
      - write_pytest_unit_tests
      - write_pytest_integration_tests
      - write_cypress_e2e_tests
      - write_k6_load_tests
      - write_contract_tests
      - open_pr_tests

history: []  # Append-only. Format: {timestamp, agent, action, ref, sha}
```

STEP 2 — SCAFFOLD REPOSITORY

Create the following directory structure in one `git commit` before any agent begins coding. Commit message: `chore: initial repo scaffold [orchestrator]`.

```
bankoffer-ai/
├── audit.yaml                          # Single source of truth (YOU ARE
HERE)
├── CLAUDE.md                           # This prompt file
├── README.md
├── .github/
|   ├── workflows/
|   |   ├── ci.yaml                     # Lint, test, build on PR
|   |   ├── cd-staging.yaml             # Deploy to staging on merge to main
|   |   ├── cd-prod.yaml                # Deploy to prod on tag
|   |   ├── agent-infra.yaml            # Triggered by repository_dispatch:
infra_agent
|   |   ├── agent-data.yaml             # Triggered by repository_dispatch:
data_agent
|   |   ├── agent-aiml.yaml             # Triggered by repository_dispatch:
aiml_agent
|   |   ├── agent-api.yaml              # Triggered by repository_dispatch:
api_agent
|   |   ├── agent-notifications.yaml    # Triggered by repository_dispatch:
notification_agent
|   |   ├── agent-gitops-sub.yaml       # Triggered by repository_dispatch:
gitops_sub
|   |   ├── agent-security-sub.yaml     # Triggered by repository_dispatch:
security_sub
|   |   ├── agent-observability-sub.yaml
|   |   └── agent-testqa-sub.yaml
|   ├── CODEOWNERS
|   └── pull_request_template.md
|
├── infra/
|   ├── terraform/
|   |   ├── main.tf
|   |   ├── variables.tf
```

```
│   │   ├── outputs.tf
│   │   └── modules/
│   │       ├── eks/
│   │       ├── rds/
│   │       └── kafka/
│   └── helm/
│       ├── bankoffer-api/
│       ├── bankoffer-worker/
│       └── bankoffer-frontend/
│
├── gitops/
│   ├── argocd/
│   │   ├── app-of-apps.yaml
│   │   └── applications/
│   │       ├── api.yaml
│   │       ├── worker.yaml
│   │       └── frontend.yaml
│   └── kustomize/
│       ├── base/
│       ├── overlays/
│       │   ├── staging/
│       │   └── prod/
│
├── services/
│   ├── api/                            # FastAPI backend
│   │   ├── main.py
│   │   ├── routers/
│   │   │   ├── offers.py
│   │   │   ├── profiles.py
│   │   │   └── webhooks.py
│   │   ├── models/
│   │   ├── middleware/
│   │   ├── tests/
```

```
|   |   ├── Dockerfile
|   |   └── requirements.txt
|   |
|   ├── worker/                        # Background ML scoring worker
|   |   ├── main.py
|   |   ├── profiler.py
|   |   ├── scorer.py
|   |   ├── ranker.py
|   |   ├── Dockerfile
|   |   └── requirements.txt
|   |
|   └── notification/                  # Notification dispatcher
|       ├── router.py
|       ├── adapters/
|       |   ├── push.py
|       |   ├── email.py
|       |   └── inapp.py
|       ├── preference_service.py
|       ├── Dockerfile
|       └── requirements.txt
|
├── data/
|   ├── kafka/
|   |   └── consumers/
|   ├── airflow/
|   |   └── dags/
|   ├── dbt/
|   |   ├── models/
|   |   |   ├── staging/
|   |   |   ├── intermediate/
|   |   |   └── marts/
|   |   └── dbt_project.yml
|   └── feature_store/
```

```
|       └── feature_definitions.py
|
├── ml/
|   ├── training/
|   |   ├── train_profile_model.py
|   |   └── train_scoring_model.py
|   ├── inference/
|   |   └── predict.py
|   ├── registry/
|   |   └── mlflow_config.yaml
|   └── ab_testing/
|       └── experiment_config.yaml
|
├── frontend/
|   ├── src/
|   |   ├── components/
|   |   ├── pages/
|   |   └── App.tsx
|   ├── public/
|   ├── package.json
|   └── Dockerfile
|
├── observability/
|   ├── prometheus/
|   |   └── rules.yaml
|   ├── grafana/
|   |   └── dashboards/
|   ├── alertmanager/
|   |   └── config.yaml
|   └── loki/
|       └── config.yaml
|
├── security/
```

```
|   ├── opa/
|   |   └── policies/
|   ├── .gitleaks.toml
|   └── sbom/
|
├── tests/
|   ├── unit/
|   ├── integration/
|   ├── e2e/
|   |   └── cypress/
|   └── load/
|       └── k6/
|
└── docs/
    ├── architecture.md
    ├── agent-communication.md
    └── runbook.md
```

STEP 3 — AGENT COMMUNICATION PROTOCOL

All agents communicate via GitHub. The protocol is strict and auditable.

Orchestrator → Depth-1 Agent (dispatch)

```bash
# Orchestrator triggers an agent workflow via repository_dispatch
gh api repos/{ORG}/bankoffer-ai/dispatches \
  -f event_type="agent_{AGENT_NAME}" \
  -f client_payload='{"task_list": ["task1","task2"], "audit_ref": "HEAD~0", "b
```

Depth-1 Agent → Depth-2 Subagent (dispatch)

Same mechanism. Depth-1 agents may only dispatch to their assigned subagents (see `audit.yaml`). Depth-2 agents CANNOT dispatch further — max depth is 2.

Agent → Agent (async result)

Each agent opens a PR when its tasks are complete. The PR description MUST contain a YAML block:

```yaml
# agent-result
agent: infra_agent
tasks_completed:
  - provision_k8s_namespaces
  - write_terraform_modules
audit_entry:
  timestamp: "{{ISO}}"
  sha: "{{HEAD_SHA}}"
  status: complete
```

The Orchestrator reads merged PRs and appends the `audit_entry` to `audit.yaml`'s `history` section, then commits directly to `main`.

Mandatory audit append (after every completed task)

```python
```

```python
# Every agent must call this before opening its PR
import yaml, datetime, subprocess

def append_audit(agent_name: str, action: str, ref: str):
    sha = subprocess.check_output(["git","rev-parse","HEAD"]).decode().strip()
    with open("audit.yaml","r") as f:
        doc = yaml.safe_load(f)
    doc["history"].append({
        "timestamp": datetime.datetime.utcnow().isoformat() + "Z",
        "agent": agent_name,
        "action": action,
        "ref": ref,
        "sha": sha
    })
    with open("audit.yaml","w") as f:
        yaml.dump(doc, f, default_flow_style=False, sort_keys=False)
    subprocess.run(["git","add","audit.yaml"])
    subprocess.run(["git","commit","--amend","--no-edit"])
```

STEP 4 — AGENT TASK SPECIFICATIONS

INFRA AGENT (depth 1, max 6 tasks)

Branch: `agent/infra`

#	Task	Output file(s)
1	`provision_k8s_namespaces`	`infra/terraform/modules/eks/namespac`
2	`write_terraform_modules`	`infra/terraform/main.tf` + modules
3	`write_helm_charts`	`infra/helm/bankoffer-api/`
4	`configure_argocd_app_of_apps`	`gitops/argocd/app-of-apps.yaml`
5	`write_github_actions_infra_workflow`	`.github/workflows/agent-infra.yaml`
6	`open_pr_infra`	PR #infra → main

After task 4, dispatch `gitops_sub` and `observability_sub` via `repository_dispatch`.

DATA PIPELINE AGENT (depth 1, max 6 tasks)

Branch: `agent/data`

#	Task	Output
1	`write_kafka_consumer`	`data/kafka/consumers/transaction_consumer.p`
2	`write_transaction_normalizer`	`data/kafka/consumers/normalizer.py`
3	`write_feature_store_loader`	`data/feature_store/feature_definitions.py`
4	`write_airflow_dags`	`data/airflow/dags/customer_profile_dag.py`
5	`write_dbt_models`	`data/dbt/models/marts/customer_features.sql`
6	`open_pr_data`	PR #data → main

AI/ML AGENT (depth 1, max 6 tasks)

Branch: `agent/aiml`

#	Task	Output	Done s
1	`write_customer_profiler`	`services/worker/profiler.py`	Return `Custo` `risk_s`
2	`write_product_scorer`	`services/worker/scorer.py`	Scores for a giv
3	`write_offer_ranker`	`services/worker/ranker.py`	Return confide
4	`write_model_registry_config`	`ml/registry/mlflow_config.yaml`	MLflow configu
5	`write_ab_test_config`	`ml/ab_testing/experiment_config.yaml`	Contro
6	`open_pr_aiml`	PR #aiml → main	PR ope

Core logic: The profiler ingests last 12 months of transactions + demographics. It classifies `life_stage` (new_graduate, young_family, pre_retirement, etc.) and computes `risk_tolerance` (1–10). The scorer maps the profile to each product in the bank catalog using a lightweight LLM call to `claude-sonnet-4-20250514` with a structured output prompt.

API AGENT (depth 1, max 6 tasks)

Branch: `agent/api`

#	Task	Output	Done signal
1	`write_fastapi_app`	`services/api/main.py`	`uvicorn` starts w error
2	`write_auth_middleware`	`services/api/middleware/auth.py`	JWT validation wo
3	`write_offer_endpoint`	`services/api/routers/offers.py`	`GET /offers/{custo` returns ranked list
4	`write_webhook_receiver`	`services/api/routers/webhooks.py`	Receives bank tran events
5	`write_openapi_spec`	`docs/openapi.yaml`	Valid OpenAPI 3.1
6	`open_pr_api`	PR #api → main	PR opened with ag block

After task 4, dispatch `security_sub` and `test_qa_sub` via `repository_dispatch`.

NOTIFICATION AGENT (depth 1, max 6 tasks)

Branch: `agent/notifications`

#	Task	Output	I s
1	`write_notification_router`	`services/notification/router.py`	I c I
2	`write_push_adapter`	`services/notification/adapters/push.py`	I A i
3	`write_email_adapter`	`services/notification/adapters/email.py`	S i
4	`write_in_app_adapter`	`services/notification/adapters/inapp.py`	V - I
5	`write_preference_service`	`services/notification/preference_service.py`	I c c f I
6	`open_pr_notifications`	PR #notifications → main	I V r h

Notification payload must include: `offer_id`, `product_name`, `personalization_reason` (one sentence from the ranker), `cta_url`, `channel`, `customer_id`.

STEP 5 — DEPTH-2 SUBAGENT SPECS

GITOPS SUBAGENT (depth 2, parent: infra_agent, max 6 tasks)

Branch: `sub/gitops`

1. `write_argocd_application_crds` — one Application CRD per service in `gitops/argocd/applications/`
2. `write_kustomize_overlays` — staging and prod overlays with image tag patching
3. `configure_image_updater` — ArgoCD Image Updater annotation on each Application CRD
4. `write_rollout_strategy` — Argo Rollouts `Rollout` resource with canary steps (10% → 30% → 100%)
5. `validate_gitops_manifests` — run `kubectl --dry-run=client` on all manifests; must pass
6. `open_pr_gitops` — PR with agent-result block, tagging infra_agent as reviewer

SECURITY SUBAGENT (depth 2, parent: api_agent, max 6 tasks)

Branch: `sub/security`

1. `run_gitleaks_scan` — scan repo for leaked secrets; fail if any found
2. `run_trivy_image_scan` — scan all Dockerfiles; write report to `security/sbom/trivy-report.json`
3. `write_opa_policies` — RBAC + data-access policies in `security/opa/policies/`
4. `generate_sbom` — CycloneDX SBOM for all services in `security/sbom/`

5. `write_secret_rotation_job` — Kubernetes CronJob that rotates DB credentials via Vault
6. `open_pr_security` — PR with agent-result block

OBSERVABILITY SUBAGENT (depth 2, parent: infra_agent, max 6 tasks)

Branch: `sub/observability`

1. `write_prometheus_rules` — alert on `offer_scoring_latency_p99 > 500ms` and `notification_delivery_failure_rate > 1%`
2. `write_grafana_dashboards` — JSON dashboard: offer acceptance rate, profile coverage, notification funnel
3. `write_alertmanager_config` — PagerDuty + Slack routing rules
4. `instrument_fastapi_metrics` — add `prometheus_fastapi_instrumentator` to `services/api/main.py`
5. `write_loki_config` — structured JSON logging for all services, ship to Loki
6. `open_pr_observability` — PR with agent-result block

TEST/QA SUBAGENT (depth 2, parent: api_agent, max 6 tasks)

Branch: `sub/testqa`

1. `write_pytest_unit_tests` — unit tests for `profiler.py`, `scorer.py`, `ranker.py` (≥80% coverage)
2. `write_pytest_integration_tests` — end-to-end API tests against test Postgres
3. `write_cypress_e2e_tests` — dashboard smoke test: login → view offer → click CTA
4. `write_k6_load_tests` — load test `GET /offers/{id}` at 500 RPS for 60s; p99 < 300ms
5. `write_contract_tests` — Pact contract tests between API and notification service

6. `open_pr_testqa` — PR with agent-result block

STEP 6 — GITOPS ARGOCD SYNC POLICY

Every service deployment is managed by ArgoCD. No `kubectl apply` is ever run manually.

```yaml
```

```yaml
# gitops/argocd/applications/api.yaml (template)
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: bankoffer-api
  namespace: argocd
  annotations:
    argocd-image-updater.argoproj.io/image-list: api=ghcr.io/{{ORG}}/bankoffer-
    argocd-image-updater.argoproj.io/api.update-strategy: semver
spec:
  project: default
  source:
    repoURL: https://github.com/{{ORG}}/bankoffer-ai
    targetRevision: HEAD
    path: gitops/kustomize/overlays/prod
  destination:
    server: https://kubernetes.default.svc
    namespace: bankoffer-prod
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
```

STEP 7 — CI PIPELINE (all agents write to this collectively)

`.github/workflows/ci.yaml` must include:

- `ruff` lint + `mypy` type-check (Python services)

- `pytest` unit tests with coverage gate (≥80%)
- `helm lint` all charts
- `terraform validate`
- `docker build --no-cache` (build but do not push on PR)
- `gitleaks detect` secret scan
- `trivy fs .` vulnerability scan
- OpenAPI spec validation via `spectral lint`
- audit.yaml schema validation (must match the defined schema above)

STEP 8 — ORCHESTRATOR EXECUTION ORDER

Execute in this exact sequence. Update `audit.yaml` after each step.

```
1. Install MCPs (Step 0)
2. Create audit.yaml (Step 1)
3. Scaffold repo (Step 2)
4. git init + initial commit + push to GitHub
5. Dispatch infra_agent      → branch agent/infra
6. Dispatch data_pipeline_agent → branch agent/data
7. Dispatch aiml_agent       → branch agent/aiml
8. Dispatch api_agent        → branch agent/api
9. Dispatch notification_agent → branch agent/notifications
10. Monitor PR queue; merge in dependency order:
    data → aiml → api → notifications → infra
11. Infra_agent dispatches:  gitops_sub, observability_sub
12. Api_agent dispatches:    security_sub, test_qa_sub
13. Monitor sub PRs; merge when CI passes
14. Final: verify ArgoCD sync status = Healthy on staging
15. Append final audit entry to audit.yaml and push
```

CONSTRAINTS & RULES (read before every action)

Rule	Detail
Audit-first	Read `audit.yaml` before every action. Never act on stale state.
Append-only history	Never delete or modify existing `history` entries in `audit.yaml`.
GitHub-only comms	Agents communicate ONLY via PRs, Issues, and `repository_dispatch`. No direct function calls between agents.
Max depth = 2	Depth-2 subagents CANNOT spawn further agents. Violation = halt.

Rule	Detail
Max 6 tasks/agent	If a task is too large, split into a new subagent (requires Orchestrator approval via Issue).
GitOps only	No `kubectl apply` or direct cloud API calls from agents. All changes via ArgoCD sync.
Branch naming	`agent/{name}` for depth-1, `sub/{name}` for depth-2. Never commit to `main` directly except for `audit.yaml` appends.
PR template	Every PR must include the `# agent-result` YAML block (see Step 3).
Secret hygiene	No secrets in code. All secrets via Kubernetes Secrets + external-secrets-operator pulling from Vault.
Claude API calls	The `aiml_agent` scorer uses `claude-sonnet-4-20250514` with `max_tokens: 1000`. Model string is hardcoded — do not change.

TECH STACK REFERENCE

Layer	Technology
Orchestration	ArgoCD + Argo Rollouts
Infrastructure	Terraform (EKS / GKE)
Packaging	Helm + Kustomize
CI	GitHub Actions
API	FastAPI + Pydantic v2

Layer	Technology
ML Scoring	Claude `claude-sonnet-4-20250514` via Anthropic API
Feature Store	Feast
Streaming	Apache Kafka
Transforms	dbt + Airflow
Database	PostgreSQL (RDS) + Redis (cache)
Notifications	FCM (push) · SendGrid (email) · WebSocket (in-app)
Frontend	React + TypeScript + Tailwind
Observability	Prometheus + Grafana + Loki + Alertmanager
Security	OPA + Gitleaks + Trivy + CycloneDX SBOM
Secret mgmt	HashiCorp Vault + external-secrets-operator
Testing	pytest · Cypress · k6 · Pact

START COMMAND

When you are ready, say:

> **"Begin execution. Reading audit.yaml."**

Then proceed through Steps 0–8 in order without pausing unless `audit.yaml` shows a blocking dependency or a CI check fails.